

Семинално упражнение 2: Софтуерни шаблони. Factory pattern. Dependency injection. Работа с абстрактни класове и интерфейси.

инж. Лилия Андонова
ТУ – София
ФКСТ

Упражнението днес:

- Какво са класове и обекти
- Софтуерни шаблони.
- Factory pattern.
- Dependency injection.
- Работа с абстрактни класове и интерфейси.

Какво е клас?

- Класът е основен елемент на обектно-ориентираното програмиране (ООП) и представлява шаблон или схема, по която се създават обекти.
- Класът определя какви атрибути (полетата/променливите) и методи (функции) ще има обектът.
- В примера класът Car представлява шаблон за автомобили с полета като **model, year, и color** и методи като **startEngine() и displayInfo().**

```
public class Car {    // Полета на класа (атрибути)
    String model;
    int year;
    String color;
// Метод на класа
    public void startEngine() {
    System.out.println("Engine started.");
    }
    public void displayInfo() {
    System.out.println("Model: " + model + ", Year: "
+ year + ", Color: " + color);
    }
}
```

Какво е обект?

- Обектът е конкретна инстанция на даден клас. С други думи, когато създадете обект от даден клас, вие създавате реализация на този клас с конкретни стойности за неговите полета.

```
public class Main {
    public static void main(String[] args) {
    // Създаване на обект от класа Car
        Car myCar = new Car();
        myCar.model = "Toyota Corolla";
        myCar.year = 2020;
        myCar.color = "Red";
    // Извикване на метод
        myCar.displayInfo(); // Output: Model:
        Toyota Corolla, Year: 2020, Color: Red
        myCar.startEngine(); // Output: Engine started.
    }
}
```

Конструктори

- Конструкторите са специални методи, които се използват за инициализация на обекти. Те имат същото име като класа и нямат върнат тип.
- Конструкторът се извиква автоматично при създаването на нов обект с ключовата дума new.
- Инициализира обекта и задава първоначални стойности на неговите променливи или извършва начална настройка.

```
public class Car {
    String model;
    int year;
    String color;
// Конструктор с параметри
    public Car(String model, int year, String color) {
        this.model = model;
        this.year = year;
        this.color = color;
    }
    ..................................................
}
```

Разлики между клас и обект

- Клас: Абстрактна концепция или шаблон, който описва какво представлява дадена структура (напр. какво съдържа и какво може да прави).
- Обект: Конкретна инстанция на класа с реални стойности за неговите атрибути.

Полета и методи на класа

- Полета: Променливи, които съхраняват състоянието на обекта (напр. model, year).
- Методи: Функции, които описват поведението на обекта (напр. startEngine()).

```
public class Calculator {
    public int add(int a, int b) {
        return a + b;
    }
}
// Тест в главния клас
public class Main {
    public static void main(String[] args) {
        Calculator calc = new Calculator();
        int result = calc.add(5, 3);
        System.out.println("Sum: " + result); // Output: Sum: 8
    }
}
```

Достъп до членовете на класа

- Модификаторите за достъп определят видимостта на полетата и методите:

модификатор	клас	пакет	наследници	проект
public	да	да	да	да
protected	да	да	да	не
private	да	не	не	не
no modifier	да	да	не	не

Капсулация (Encapsulation)

Капсулация е принцип в ООП, който включва комбинирането на данни (променливи) и методи в един обект, така че те да бъдат достъпни и манипулирани само по определен начин.

В основата си капсулацията цели да защити вътрешните данни на обекта от външни промени или неоторизиран достъп и да осигури интерфейс за безопасно взаимодействие с тези данни.

Основни аспекти на капсулацията:

- Скриване на данни (data hiding): Частните (private) променливи на класа не могат да бъдат достъпвани директно извън класа.
- Контрол на достъпа: Чрез публични (public) методи за достъп (например get и set методи), можем да контролираме как данните се променят и четат.

```
public class BankAccount {
    // Частна променлива
    private double balance;
// Публичен конструктор
    public BankAccount(double initialBalance) {

    if (initialBalance >= 0) {
        this.balance = initialBalance;
    } else {
        this.balance = 0;           }    }
// Публичен метод за депозиране на пари
    public void deposit(double amount) {
        if (amount > 0) {
            balance += amount;      }    }
// Публичен метод за теглене на пари
     public void withdraw(double amount) {
        if (amount > 0 && amount <= balance) {
            balance -= amount;      }    }
// Публичен метод за получаване на баланса
    public double getBalance() {      return balance;    }}
```

Променливата balance е частна (private), което означава, че не може да бъде достъпвана или променяна директно от други класове.

Методи като deposit(), withdraw(), и getBalance() предоставят контролирани пътища за взаимодействие с тази частна променлива.

Така капсулацията осигурява интерфейс за манипулиране на данните, като едновременно с това предпазва данните от некоректни стойности или неоторизирани промени.

Наследяване

Наследяването е концепция от обектно-ориентираното програмиране (ООП), при която един клас (наречен дъщерен клас) може да наследява свойства и методи от друг клас (наречен родителски клас или базов клас).

Това позволява на програмите да използват код, който вече е дефиниран в родителския клас, без да е необходимо да се пише отново.

Основни понятия:

- Родителски клас (Super class): Класът, от който се наследява.
- Дъщерен клас (Sub class): Класът, който наследява от родителския клас.

В Java наследяването се реализира чрез ключовата дума *extends*.

Какво се наследява?

```
class Animal {
    void eat() {
        System.out.println("This animal eats food.");    }}
class Dog extends Animal {
    void bark() {
        System.out.println("The dog barks.");    }}
    public class Main {
    public static void main(String[] args) {
    Dog dog = new Dog();
    dog.eat();  // Методът "eat" се наследява от Animal
    dog.bark(); // Методът "bark" е дефиниран в Dog    }}
```

Методи: Дъщерният клас наследява публичните и защитените методи на родителския клас.

Полета (параметри): Дъщерният клас наследява полетата на родителския клас, но не може да ги достъпва директно, ако те са частни. За достъп се използват публични или защитени гетъри и сетъри.

Конструктори: Конструкторите не се наследяват, но дъщерният клас може да извика конструктор на родителския клас с помощта на ключовата дума super.

```
class Dog extends Animal {
    Dog() {       // Конструткор на класа Dog
        super(); // Извиква конструктора на Animal
        System.out.println("A dog is created.");
}
```

Софтуерни шаблони (Design Patterns)

- Софтуерни шаблони (Design Patterns) са установени решения на повтарящи се проблеми в софтуерното проектиране.
- Вместо да решаваме проблемите от нулата всеки път, можем да използваме шаблони, които предоставят готова структура, която да следваме.
- Те са особено полезни за проектиране на гъвкав и мащабен код, като осигуряват решения за проблеми, които често срещаме при създаването на различни видове софтуерни приложения.
- Типове шаблони:
- **Шаблони за създаване на обекти** (Creational Patterns): Помагат при създаването на обекти. Пример: Factory Pattern.
- **Структурни шаблони** (Structural Patterns): Помагат при организирането и свързването на класовете. Пример: Adapter.
- **Поведенчески шаблони** (Behavioral Patterns): Описват взаимодействието между обектите. Пример: Observer.

Factory Pattern

- Factory Pattern се използва за създаване на обекти, като предоставя интерфейс за създаването им, без да разкрива детайлите за тяхната имплементация. Целта на този шаблон е да избегнем твърдо кодиране (hard coding) на конкретни класове при създаването на обекти и вместо това да използваме единен начин за създаване на различни обекти.
- Полезен е, когато имаме нужда да създаваме обекти, базирани на параметри (например тип или специфични свойства), които определят кой точно клас да се използва.
- Фабричният клас е специален клас, който съдържа метод (или методи), отговорни за създаването на обекти. Той решава коя конкретна имплементация на даден интерфейс или абстрактен клас да създаде в зависимост от подадените параметри или условия.

```
interface Car {
    void drive();
}
class Sedan implements Car {
    public void drive() {
        System.out.println("Driving a Sedan");
    }
}
class SUV implements Car {
    public void drive() {
        System.out.println("Driving an SUV");
    }
}
class Truck implements Car {
    public void drive() {
        System.out.println("Driving a Truck");    }}
```

```
class CarFactory {
    public static Car createCar(String type) {
        switch (type) {
            case "Sedan": return new Sedan();
            case "SUV": return new SUV();
            case "Truck": return new Truck();
            default: throw new
IllegalArgumentException("Unknown car type");
        }
    }
}
// Използване
Car car = CarFactory.createCar("SUV");
car.drive();  // Output: Driving an SUV
```

Dependency Injection (DI)

Dependency Injection (DI) е техника за предоставяне на зависимости на обект, вместо той сам да ги създава. Това прави класовете по-гъвкави и лесни за тестване, тъй като можем да „внедрим“ зависимости с различни имплементации без да променяме основния клас.

Предимства: Dependency Injection осигурява възможност за тестване с фалшиви обекти и улеснява промяната на класовете, когато има нови зависимости.

Видове DI:

- Конструкторна инжекция: Зависимостите се предават чрез конструктора на класа.
- Методна инжекция: Зависимостите се предават чрез setter методи.
- Полева инжекция: Зависимостите се предават чрез директно присвояване на полета (често се използва в Spring).

Dependency Injection (DI) чрез конструктор

```
class Bulb {
    void shine() {
        System.out.println("Крушката свети");
    } }
class Lamp {
    private Bulb bulb;
    Lamp(Bulb bulb) {
        this.bulb = bulb;
    } void turnOn() {
        bulb.shine();
    } }
Bulb bulb = new Bulb();
Lamp lamp = new Lamp(bulb);
lamp.turnOn();
```

- клас за крушка
- В конструктора Lamp(Bulb bulb) подаваме крушка на лампата.
- Създаваме крушката и я подаваме на лампата

Dependency Injection (DI) чрез метод

```
class Bulb {
    void shine() {
        System.out.println("Крушката свети");
    }}
class Lamp {
    private Bulb bulb;
    void setBulb(Bulb bulb) {
        this.bulb = bulb;
    }
    void turnOn() {
        bulb.shine();
    }}
public class Main {
    public static void main(String[] args) {
        Bulb bulb = new Bulb();   // създаваме крушката
        Lamp lamp = new Lamp();   // създаваме лампата
        lamp.setBulb(bulb);       // подаваме крушката чрез метод
        lamp.turnOn();
    }}
```

- Разликата спрямо подаването през конструктора е, че първо създаваме лампата, а после ѝ подаваме крушката с setBulb(bulb). Методът setBulb е мястото, където лампата получава зависимостта си.

Абстрактни класове

- Това е клас, който може да съдържа както абстрактни методи (без тяло), така и методи с имплементация.
- Не може да бъде инициализиран директно. Абстрактните класове са полезни, когато имаме основни методи или свойства, които всички подкласове трябва да наследят.
- Абстрактен клас не може да се инстанцира! Може само да се наследява.
- Ако даден клас наследи абстрактен клас, то той задължително трябва да имплементира абстрактните му методи или и той трябва да е абстрактен, ако остави дори и един от тях абстрактен.
- *public abstract class NameClass{*

  public void someMethod(){

  ……..

  }

  public abstract void nameAbstractMethod();

}

Интерфейси

- Интерфейс не може да се инстанцира.
- В интерфейс може да има абстрактни и default методи.
- Интерфейс може да бъде имплементиран само от клас.
- За да може клас да имплементира интерфейс, то интерфейсът трябва да е публичен, методите му също.
- Интерфейсът не може да бъде деклариран с private, protected или transient.
- По подразбиране всички методи на интерфейса са: public abstract.
- Променливите на интерфейса са public static final.
- Деклариране на интерфейс: Става с ключовата дума interface:

```
public interface NameInterface{
        public abstract void method1();
        public abstract void method2();
}
```

Декларация на интерфейс

```
[модификатор_за_достъп] interface ИмеНаИнтерфейс {
    // 1. Константи (по желание)
    [public static final] тип име = стойност;
    // 2. Абстрактни методи (задължителни за имплементация)
    [public abstract] тип_връщане имеМетод(параметри);
    // 3. Default методи (с тяло) – от Java 8 нататък
    default тип_връщане имеМетод(параметри) {
        // реализация
    }
    // 4. Static методи – от Java 8 нататък
    static тип_връщане имеМетод(параметри) {
        // реализация
    }
}
```

interface – ключова дума за създаване на интерфейс

модификатор_за_достъп – обикновено е **public** или се пропуска (тогава е достъпен само в пакета)

константи – всички са по подразбиране public static final

абстрактни методи – всички са по подразбиране public abstract. Извикват се чрез обект

default методи – имат тяло; могат да се променят в класа. Извикват се чрез обект

static методи – извикват се директно чрез името на интерфейса и не се променят в класа

Обекти – не могат да се създават от интерфейс

Къде декларираме интерфейс

Място на интерфейса	Къде се използва	Особености
В отделен файл (MyInterface.java)	Навсякъде в проекта	Най-често срещаният и препоръчителен вариант
Като вътрешен интерфейс в клас	Само вътре в класа	Подходящ за специализирани случаи
В същия файл с други класове	В малки програми или примери	Само един елемент в файла може да е public

Полета на интерфейс

- Константите са стойности, които не се променят по време на изпълнение.
- В интерфейсите всички променливи са по подразбиране public static final.
- Използват се за фиксирани параметри, които са общи за всички класове, имплементиращи интерфейса.

Имплементация на интерфейс

- *Имплементиране (implements) на интерфейс означава даден клас да реализира всички негови методи.*
- **Синтаксис:**

```
class ИмеНаКлас implements ИмеНаИнтерфейс {
    // тук се пишат реализациите на методите
}
```

Имплементиране на няколко интерфейса

Java позволява множествена имплементация — един клас може да имплементира повече от един интерфейс.

```
interface Flyable {    void fly();}
interface Swimmable {    void swim();}
class Duck implements Flyable, Swimmable {
    @Override
    public void fly() {
        System.out.println("Патицата лети.");
    }
    @Override
    public void swim() {
        System.out.println("Патицата плува.");
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        Duck d = new Duck();
        d.fly();
        d.swim();
    }
}
```

Имплементиране + Наследяване

Клас може да наследява друг клас и едновременно да имплементира интерфейси.

```
class Animal {
    void eat() {
        System.out.println("Животното яде.");
    }
}
interface Runnable {
    void run();
}
class Dog extends Animal implements Runnable {
    @Override
    public void run() {
        System.out.println("Кучето тича.");
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.eat(); // метод от родителския клас Animal
        d.run(); // метод от интерфейса Runnable
    }
}
```

Обощено за интерфейсите

- Интерфейсът описва какво трябва да може даден клас, но не казва как се прави това.
- Интерфейс не може да се инстанцира.
- В интерфейс може да има абстрактни и default методи.
- Интерфейс може да бъде имплементиран само от клас.
- За да може клас да имплементира интерфейс, то интерфейсът трябва да е публичен, методите му също.
- Интерфейсът не може да бъде деклариран с private, protected или transient.
- По подразбиране всички методи на интерфейса са: public abstract.
- Променливите на интерфейса са public static final.
- Деклариране на интерфейс: Става с ключовата дума interface:

Какво научих днес:

- Какво са класове и обекти
- Софтуерни шаблони.
- Factory pattern.
- Dependency injection.
- Работа с абстрактни класове и интерфейси.